

ML Training Data Guide

Dispotree Deal Quality Prediction Model

Generated: 12/28/2025

Overview

The Dispotree ML model predicts deal success probability - whether a deal will be accepted by a fund and close successfully. It uses an 85-dimension feature vector.

Minimum Requirements

- Minimum samples: 100 (model won't train with fewer)
- Recommended samples: 500+ for better accuracy
- Positive/Negative ratio: 30-70% positive
- Required fields: 14 columns minimum

85-Dimension Feature Vector

- 9 Numeric features: price, ARV, sqft, year, beds, baths, photos, repair, rent
- 6 Derived features: price/sqft, ARV ratio, equity %, age, repair ratio, rent ratio
- 50 One-hot state features (US states)
- 10 One-hot property type features
- 5 Buy box match features
- 16 Image analysis features

Required CSV Columns

Core Deal Information (REQUIRED)

These fields are essential for the model to make predictions:

deal_id	string	"deal-001"	Unique identifier
asking_price	number	185000	Current asking price (\$)
arv	number	265000	After Repair Value (\$)
sqft	number	1850	Square footage
year_built	number	1998	Year built
bedrooms	number	3	Bedroom count
bathrooms	number	2.5	Bathroom count
repair_estimate	number	35000	Rehab cost (\$)
state	string	"FL"	2-letter state code
property_type	string	"single_family"	Property type

Fund Response (REQUIRED FOR TRAINING LABELS)

These fields determine if a deal was successful (training labels):

fund_name	string	"Tricon"	Fund that received deal
buy_box_id	string	"bb-001"	Buy box ID used for scoring
submission_score	number	87.5	Your score when submitted (0-100)
response_type	string	"closed"	Fund's response (see below)
offer_amount	number	178000	Fund's offer amount (if made)

Response Types (Training Labels)

POSITIVE (Label = 1):

- closed - Deal closed successfully
- offer_accepted - Offer accepted, heading to close

NEGATIVE (Label = 0):

- passed - Fund passed on deal
- no_response - No response after 14+ days

NEUTRAL (Wait for outcome):

- offer_made - Fund made offer (wait for acceptance)
- interested - Fund expressed interest (wait for action)

Optional Fields

Location Details (RECOMMENDED)

street	string	"123 Oak Lane"	Street address
city	string	"Orlando"	City name
zip	string	"32801"	ZIP code

Market Data (RECOMMENDED)

monthly_rent	number	1800	Expected monthly rent (\$)
days_on_market	number	14	Days since listing
photo_count	number	12	Number of listing photos
occupancy_status	string	"vacant"	vacant, occupied, unknown
condition	string	"good"	excellent, good, fair, poor

Win/Loss Analysis (HIGHLY RECOMMENDED)

This data helps the model learn WHY deals succeed or fail:

loss_category	string	"price"	Why deal was lost
win_factors	string	"quick_close"	Success factors (semicolon-separated)
lessons_learned	string	"text"	Notes for future reference

Loss Categories:

price, condition, location, competition, timing, financing, other

Win Factors (examples):

quick_close, good_comps, below_market_rent, tenant_quality, newer_build, low_rehab, good_schools, premium_location, turnkey

Image Features (OPTIONAL)

image_quality_score	number	75	Overall quality (0-100)
image_condition	string	"good"	excellent, good, fair, poor
has_updated_kitchen	0/1	1	Kitchen appears updated
has_hardwood	0/1	1	Has hardwood floors
has_natural_light	0/1	1	Good natural lighting
has_damage	0/1	0	Visible damage in photos
needs_repair	0/1	0	Obvious repairs needed
is_dated	0/1	0	Appears dated

Valid Values Reference

Property Types

single_family, condo, townhouse, multi_family, duplex, triplex, quadplex, manufactured, land, other

US State Codes

AL, AK, AZ, AR, CA, CO, CT, DE, FL, GA, HI, ID, IL, IN, IA, KS, KY, LA, ME, MD, MA, MI, MN, MS, MO, MT, NE, NV, NH, NJ, NM, NY, NC, ND, OH, OK, OR, PA, RI, SC, SD, TN, TX, UT, VT, VA, WA, WV, WI, WY, DC

Derived Features (Auto-Calculated)

The model automatically calculates these from your data:

Price per sqft	= asking_price / sqft
ARV to price ratio	= arv / asking_price
Equity percentage	= ((arv - asking_price) / arv) * 100
Property age	= current_year - year_built
Repair to price	= repair_estimate / asking_price
Rent to price	= (monthly_rent * 12) / asking_price

Model Performance Targets

After training, aim for these metrics:

Accuracy	> 70%	Overall correctness
Precision	> 65%	True positives / predicted positives
Recall	> 60%	True positives / actual positives
F1 Score	> 62%	Harmonic mean of precision/recall
AUC	> 0.70	Area under ROC curve

Training Workflow

Step 1: Collect Data

For every deal you send to a fund, record all deal details plus fund name and your submission score.

Step 2: Track Responses

Update response_type when the fund responds:

- They pass !' "passed"
- They make offer !' "offer_made"
- You accept offer !' "offer_accepted"
- Deal closes !' "closed"
- No response in 14 days !' "no_response"

Step 3: Analyze Outcomes

- For passed deals: Record loss_category (why they passed)
- For closed deals: Record win_factors (what made it successful)

Step 4: Train the Model

Once you have 100+ samples, train via API, Agent, or CLI:

```
# API
POST /api/ml/train
{"minSamples": 100, "validationSplit": 0.2, "epochs": 100}

# Agent Command
"Train the ML model with the latest feedback data"

# CLI
npm run train:ml
```

Import Training Data

```
curl -X POST http://localhost:3001/api/ml/training/import \
-F "file=@backend/data/ml_training_data.csv"
```

Example Training Data

Positive Example (Closed Deal)

```
deal_id,asking_price,arv,sqft,year_built,bedrooms,bathrooms,state,property_type
deal-001,185000,265000,1850,1998,3,2,FL,single_family

repair_estimate,monthly_rent,fund_name,submission_score,response_type,offer_amount
35000,1800,Tricon Residential,87.5,closed,178000

win_factors,lessons_learned
quick_close;good_comps,Great location near schools helped close fast
```

Negative Example (Passed Deal)

```
deal_id,asking_price,arv,sqft,year_built,bedrooms,bathrooms,state,property_type
deal-002,145000,195000,1200,1985,2,1,FL,single_family

repair_estimate,monthly_rent,fund_name,submission_score,response_type
25000,1200,FirstKey Homes,72.3,passed

loss_category,lessons_learned
price,Price too high for condition - should have negotiated harder
```

Data Quality Tips

- Include both wins AND losses
- Be honest about loss reasons
- Include geographic diversity
- Update outcomes when deals close
- Don't only log winners
- Don't leave ARV empty
- Don't use inconsistent state codes
- Don't forget to update response outcomes

File Locations

```
Example CSV:  /backend/data/ml_training_example.csv
Your data:    /backend/data/ml_training_data.csv
This PDF:     /backend/data/ML_Training_Data_Guide.pdf
```


